

PRUEBA TÉCNICA QA

COMPONENTE PRÁCTICO (Funcional)

1) Redactar las siguientes consultas en SQL

STUDENTS

STUDENT_ID	NAME	BIRTHDATE	GENDER	EMAIL	PHONE
9000185	PEDRO GOMEZ	1985-10-30	M	PEDRO.GOMEZ@GMAIL.COM	1185432478
9000186	JUAN HERNANDEZ	1995-01-25	M	Juan.Hernandez@Gmail.com	1185432479
9000187	MARIA GONZALEZ	1998-05-05	F	MARIA.GONZALEZ@HOTMAIL.COM	1185432480
9000188	PABLO SANCHEZ	2001-08-09	M	pablo.sanchez@gmail.com	1185432481
9000189	ELENA CASTRO	1972-02-18	F	elena.castro@hotmail.com	1185432482
9000190	MATIAS PEREZ	1993-11-01	M	MATIAS.PEREZ@gmail.COM	1185432483
9000191	SEBASTIAN VARGAS	2000-09-06	M	SEBASTIAN.VARGAS@OUTLOOK.COM	1185432484
9000192	PEDRO HERNANDEZ	1998-04-22	M	PEDRO.HERNANDEZ@GMAIL.COM	1185432485
9000193	CAROLINA LUNA	1997-07-21	F	CAROLINA.LUNA@YAHOO.COM	1185432486
9000194	MONICA FIGUEROA	1987-05-06	F	MONICA.FIGUEROA@HOTMAIL.COM	1185432487

CAREERS

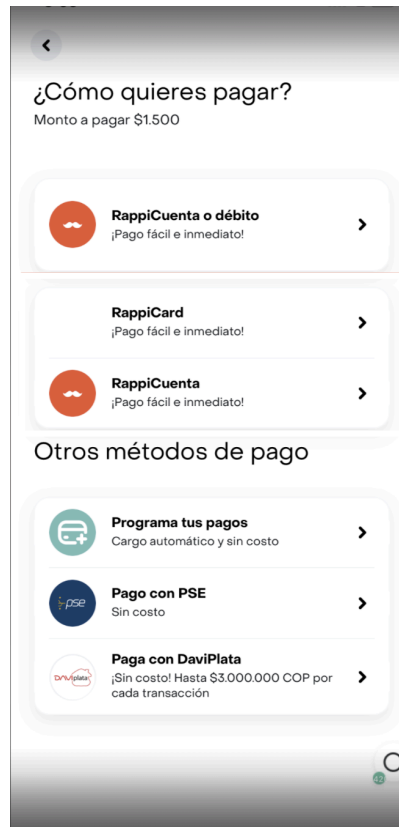
CAREER_ID	NAME	DESCRIPTION	SUBJECTS	COST
10025	INGENIERIA	INGENIERIA	90	25000,00
10026	ADMINISTRACION	ADMINISTRACION	72	18000,00
10027	CONTABILIDAD	CONTABILIDAD	60	15000,00
10028	DERECHO	DERECHO	85	23000,00
10029	INFORMATICA	INFORMATICA	78	19500,00
10030	COMUNICACION	COMUNICACION	65	17800,00

CAREER_STUDENT

CAREER_ID	STUDENT_ID	START_DATE	ESTIMATED_END_DATE	END_DATE	EXTENDED	CALIFICATION
10025	9000185	2009-01-01	2010-01-01	2010-01-01	NULL	85
10030	9000186	2008-01-01	2013-01-01	2013-07-01	NULL	90
10027	9000190	2014-01-01	2019-01-01	NULL	NULL	52
10030	9000187	2010-01-01	2015-01-01	2015-02-28	NULL	63
10028	9000191	2009-01-01	2014-01-01	2014-01-01	NULL	87
10025	9000189	2008-01-01	2013-01-01	2018-01-01	NULL	96
10026	9000191	2015-01-01	2020-01-01	NULL	NULL	85
10029	9000190	2012-01-01	2017-01-01	2017-07-05	NULL	99
10026	9000191	2016-01-01	2021-01-01	NULL	NULL	74
10026	9000192	2015-01-01	2020-01-01	NULL	NULL	72
10030	9000186	2011-01-01	2016-01-01	2016-01-01	NULL	91
10027	9000193	2012-01-01	2017-01-01	NULL	NULL	32
10029	9000185	2008-01-01	2013-01-01	2013-01-01	NULL	68

- Seleccione el total de estudiantes Masculinos Registrados
- Elimine los estudiantes nacidos antes 2000
- Actualice la descripción de la carrera con id '10026' de "Administración" a "Administración de Empresas"
- Muestre los nombres de los estudiantes que estén cursando dos o más materias
- Calificación promedio de todos los estudiantes que estén cursando carreras
- Muestra el nombre, carrera id, correos de los estudiantes que tienen una calificación mayor 70
- Cuántos estudiantes que estén cursando 2 o más carreras

2) De la siguiente imagen, reporte al menos un bug que incluya todos los datos que considere relevantes para que un desarrollador pueda replicarlo y solucionarlo.



COMPONENTE TEÓRICO (Automation)

Basado en su experiencia, responda las siguientes preguntas:

1. ¿Cuáles son las principales diferencias entre automatizar en Android vs iOS con Appium?
2. ¿Cómo diseñaría un framework de automatización para mobile multiplataforma que sea mantenible y escalable?
3. ¿Qué criterios usa para seleccionar qué casos se deben automatizar y cuáles no?
4. ¿Qué estrategias aplicaría para integrar pruebas automáticas en un pipeline CI/CD?
5. Mencione las ventajas y desventajas de trabajar con dispositivos físicos vs. dispositivos en remoto / granja de dispositivos cuando ejecuta una suite de automatización.
6. ¿Qué patrones de diseño ha implementado en sus proyectos que considere eficaces para minimizar la duplicación de código y mejorar la legibilidad al desarrollar automatizaciones para dispositivos móviles?

COMPONENTE PRÁCTICO (Automation)

Contexto: Se requiere implementar un nuevo script para una prueba automatizada. El caso de prueba involucra tres (3) vistas: **Home del App**, **Login** y el **Home de producto**. El flujo que se desea automatizar valida que el usuario Ingrese al **Home del App** donde da un tap al widget del producto para posteriormente ser redireccionado al **Login**. Allí ingresará un código de 6 dígitos y finalmente navegará de forma automática al **Home del producto** donde se encuentra el detalle y acciones que el usuario puede realizar sobre el producto.

- a) En la vista de Login se implementa un método para validar que todos los labels definidos en el diseño se encuentren presentes. A continuación verá el código implementado.

```

1  public boolean verifyLabelsOnScreen() {
2      try {
3          // Espera para que la pantalla cargue los elementos
4          Thread.sleep(15000);
5
6          boolean flagLabelIniciarSesion = false;
7          boolean flagLabelHola = false;
8          boolean flagLabelIngresarContrasena = false;
9          boolean flagLabelOlvideContrasena = false;
10
11         // Recorre todos los labels en la pantalla
12         for (MobileElement label : screenLabels) {
13             String text = label.getAttribute("text");
14
15             if (text.equals("Iniciar Sesión")) {
16                 flagLabelIniciarSesion = true;
17             }
18             if (text.contains("¡Hola!")) {
19                 flagLabelHola = true;
20             }
21             if (text.equals("Ingresa tu contraseña de 6 dígitos")) {
22                 flagLabelIngresarContrasena = true;
23             }
24             if (text.equals("Olvidé mi contraseña")) {
25                 flagLabelOlvideContrasena = true;
26             }
27         }
28
29         // Verifica que todos los labels estén presentes
30         if (flagLabelIniciarSesion && flagLabelHola
31             && flagLabelIngresarContrasena && flagLabelOlvideContrasena) {
32             return true;
33         }
34
35         } catch (Exception e) {
36             return false;
37         }
38         return false;
39     }
40

```

Instrucción: De acuerdo a su experiencia indique si está de acuerdo con la forma en la que fue implementado el método. En caso de no estar de acuerdo. Indíquenos qué modificaciones haría o reescriba el método para ver reflejados sus cambios y justifique su respuesta.

- b) En la vista del Home del App, se necesita crear dos métodos: El primero estará encargado de verificar si el usuario se encuentra o no en el Home del App y el segundo

realizará la acción de dar tap en el botón de Rappipay para ingresar al login. A continuación verá la definición de los elementos que se tuvieron en cuenta.

```
1  @AndroidFindBy(id = "com.grability.rappi:id/menu_image")
2  private MobileElement buttonMenu;
3  @AndroidFindBy(id = "com.grability.rappi:id/support_image")
4  private MobileElement buttonSupport;
5  @AndroidFindBy(className = "android.widget.TextView")
6  private List<MobileElement> widgetElements;
7  @AndroidFindBy(id = "com.grability.rappi:id/imageView_action")
8  private MobileElement buttonRappiPay;
9  @AndroidFindBy(id = "layout_primary_action")
10 private MobileElement BTN_RAPPIPAY_CO;
11 @AndroidFindBy(xpath = "//*[contains(@text, 'Restaurante')]")
12 private MobileElement RESTAURANT;
13 @AndroidFindBy(xpath = "//*[contains(@text, 'Mercado/Ex')]")
14 private MobileElement MARKET;
15 @AndroidFindBy(xpath = "//*[contains(@text, 'Turbo')]")
16 private MobileElement TURBO;
17
```

Instrucción: De acuerdo a su experiencia indique si está de acuerdo con la forma en la que se definieron los elementos. En caso de no estar de acuerdo. Indíquenos qué modificaciones haría o reescriba el código para ver reflejados sus cambios y justifique su respuesta.

COMPONENTE TÉCNICO (Automation)

Para este componente hemos definido un ejercicio el cual se detalla más adelante. Tenga en cuenta que durante la sustentación, deberá explicar y defender la solución, respondiendo a preguntas y discutiendo aspectos técnicos relevantes del proyecto.

Descripción de la Prueba

La prueba está dividida en dos partes, con un tiempo total recomendado de 8 horas, que podrás distribuir en dos días si lo prefieres.

Parte 1 – Diseño del Framework (2-3 horas)

Realiza un documento (puede ser un archivo README.md o PDF) donde explique su propuesta de arquitectura para un framework de automatización mobile que cumpla con los siguientes requisitos:

- Soporte multiplataforma: Android e iOS.
- Reutilización de código entre módulos.
- Escalabilidad para agregar futuras pruebas y funcionalidades.
- Uso de Java 11+ como lenguaje principal.
- Uso de JUnit o TestNG para el framework de testing.
- Uso de Appium para la automatización mobile.
- Gestión de dependencias con Maven o Gradle.
- Integración de logging mediante Log4j, SLF4J u otra librería similar.

Debe incluir:

- Diagrama o esquema de la estructura del proyecto.
- Explicación de los patrones de diseño y arquitectónicos que aplicarás
- Cómo manejará la configuración para diferentes plataformas.
- Estrategia para manejo de logs y reportes.
- Cómo asegura la escalabilidad y mantenibilidad.
- Cómo organizar las dependencias y ejecución de tests.

Parte 2 – Implementación del flujo automatizado (4-6 horas)

Basado en la arquitectura que diseñó, implemente un flujo de prueba automatizado que realice lo siguiente en la aplicación móvil de su preferencia (puede ser una app pública o demo disponible):

1. Abrir la aplicación.
2. Hacer login con credenciales válidas.
3. Validar que la pantalla principal (home) se ha cargado correctamente, verificando que un elemento clave (por ejemplo, título o botón visible) esté presente.
4. Cerrar sesión.

Requisitos:

- Utilice Appium para la automatización.
- Use el framework de testing seleccionado en la parte 1 (JUnit o TestNG).
- Aplique los patrones y estructura definidos en el diseño.
- Incluya logs claros que permitan trazar la ejecución.
- Maneje dependencias y ejecución mediante Maven o Gradle.

Entregables

- Código fuente completo en un repositorio público o privado de GitHub (proporcionar enlace).
- Archivo README.md que incluya:
 - Instrucciones claras para ejecutar las pruebas.
 - Detalle de las dependencias y versiones usadas.
 - Explicación breve de las decisiones de diseño arquitectónico.
 - Evidencias de ejecución del flujo automatizado con capturas de pantalla y logs de ejecución.